

their long-term earnings will be improved if they push through a local minimum to another hill to climb (their next, more valuable degree).

The local extrema problem is a serious one, and there's no silver bullet for resolving it. One way to attack the problem is to attempt multiple initial guesses and perform gradient ascent for each of them. For example, if we performed gradient ascent for 12.5, 15.5, and 18.5 years of education, we would get different results each time, and we could compare these results to see that in fact the global maximum comes from maximizing years of education (at least on this scale).

This is a reasonable way to deal with the local extremum problem, but it can take too long to perform gradient ascent enough times to get the right maximum, and we're never guaranteed to get the right answer even after hundreds of attempts. An apparently better way to avoid the problem is to introduce some degree of randomness into the process, so that we can sometimes step in a way that leads to a locally worse solution, but which in the long term can lead us to better maxima. An advanced version of gradient ascent, called *stochastic gradient ascent*, incorporates randomness for this reason, and other algorithms, like simulated annealing, do the same. We'll discuss simulated annealing and the issues related to advanced optimization in Chapter 6. For now, just keep in mind that as powerful as gradient ascent is, it will always face difficulties with the local extrema problem.

From Maximization to Minimization

So far we've sought to maximize revenue: to climb a hill and to ascend. It's reasonable to wonder whether we would ever want to go down a hill, to descend and to minimize something (like cost or error). You might think that a whole new set of techniques is required for minimization or that our existing techniques need to be flipped upside down, turned inside out, or run in reverse.

In fact, moving from maximization to minimization is quite simple. One way to do it is to “flip” our function or, more precisely, to take its negative. Going back to our tax/revenue curve example, it is as simple as defining a new flipped function like so: